Compute-Efficient Training of Small Transformer Language Models

Investigating Training Duration, Data Quantity, and Model Capacity

SONG MINGJUN
NUS Student ID: A034593Y
DSA4212 Final Project · 13 November 2026

GitHub repository: https://github.com/JS-2028/dsa4212-small-llm

---

Abstract

Training a language model under a limited compute budget requires choices about model capacity, training duration, and data quantity. This project implements a decoder-only Transformer language model from randomly initialised weights in PyTorch and investigates these choices on TinyStories using an 8,000-token byte-pair encoding tokenizer. Three exploratory studies are reported. First, a 5.27M-parameter model's validation cross-entropy decreased from 3.976 after 500 updates to 2.511 after 5,000 updates, although the runs used different learning-rate schedules and are therefore not a fully controlled duration comparison. Second, a controlled data-scaling experiment compared 10,000, 50,000, and 100,000 stories over 5,000 updates, with three random seeds per condition. Mean validation losses were 2.5555, 2.5160, and 2.5150, respectively, suggesting a benefit from the first increase in data and little additional gain from 50,000 to 100,000 stories under this training budget. Third, under an approximately 150-second wall-clock budget, models with 5.27M, 13.82M, and 29.45M parameters achieved validation losses of 2.4671, 2.6092, and 2.8401. The smallest model performed best in this hardware- and budget-specific comparison, while larger models completed fewer updates. Limitations include a single validation sample of fixed batches, a single seed for model scaling, no final test-set evaluation, and a small number of model configurations.

1. Introduction

A small language model must allocate limited computation among its parameter count, the number of optimisation updates, and the amount of training data it encounters. Increasing model size can increase representational capacity, but also raises the cost of each update. Increasing dataset size can expose a model to more diverse examples, but a fixed number of updates then covers a smaller fraction of the available corpus. Training for longer may improve predictive performance, although returns can diminish or overfitting can emerge.

This project studies these trade-offs by training a decoder-only Transformer from scratch on TinyStories. The project is motivated by compute-optimal language-model training, where model size and training-token allocation interact rather than being independent decisions [1]. TinyStories provides a controlled domain of short, simple narratives designed to study the capabilities of small language models [2]. The research question is: *How do training duration, dataset size, and model capacity affect the predictive performance of a small Transformer language model under limited computational resources?*

2. Background

Transformers use self-attention to model dependencies between sequence positions [3]. A decoder-only language model applies causal masking so each position predicts the next token using only preceding context. This implementation uses learned token and positional embeddings, pre-layer normalisation, residual connections, multi-head causal self-attention, and position-wise feed-forward networks. Byte-pair encoding represents text as subword units and can reduce dependence on a fixed word-level vocabulary [4]. Compute-optimal training research highlights the joint allocation of parameters and training tokens [1], while data-constrained training motivates examining dataset diversity and repeated exposure [5].

3. Methods

3.1 Data and preprocessing

The public TinyStories corpus contained 2,119,719 training stories and 21,990 validation stories. A deterministic split reserved 10,000 stories from the original training split as a held-out test set, leaving 2,109,719 training stories. The reserved test set was not used for training, tokenizer fitting, or the reported validation comparisons; no final test-set evaluation was completed before submission.

A byte-level BPE tokenizer with an 8,000-token vocabulary was trained on a deterministic sample of 200,000 training stories only. Special tokens were [UNK], [BOS], [EOS], and [PAD]. Each story was encoded and terminated with [EOS]. The tokenizer exactly reconstructed the inspected sample. The model context length was 256 tokens. Training batches were sampled as contiguous windows from concatenated token streams, so a window could cross a story boundary; EOS tokens mark those boundaries.

For E1's initial pilot, 10,000 stories were tokenised. E2 used nested, deterministically shuffled subsets of 10,000, 50,000, and 100,000 stories, containing 2,218,441; 11,022,217; and 22,035,205 tokens. E2 and E3 used the same fixed set of 50 validation batches (batch size 32, sequence length 256).

3.2 Architecture and optimisation

The baseline is a decoder-only Transformer implemented in PyTorch. It uses tied token-embedding and output-projection weights, learned positional embeddings, causal multi-head attention, GELU feed-forward layers, residual connections, and pre-layer normalisation. The baseline has 4 layers, embedding dimension 256, 4 attention heads, context length 256, dropout 0.1, and approximately 5.27 million trainable parameters. Larger E3 models change depth and width. All model weights were randomly initialised; no pretrained model weights, embeddings, or other pretrained neural components were used.

Training used AdamW (betas 0.9 and 0.95), weight decay 0.1, gradient clipping at 1.0, FP16 autocast and gradient scaling. The standard 5,000-update schedule used peak learning rate 3×10^{-4}, 200 warmup updates, and cosine decay. Hardware was a Google Colab NVIDIA Tesla T4 GPU with 15.64 GB reported VRAM, using PyTorch 2.14.1+cu130.

3.3 Evaluation

The primary metric is mean next-token cross-entropy on fixed validation batches; lower is better. Perplexity is exp(loss). Runtime and peak allocated GPU memory are secondary resource measures. E2 reports mean and sample standard deviation across three seeds. E3 is an exploratory single-seed comparison. Since the same tokenizer is used throughout, token-level cross-entropy and perplexity are comparable within these experiments.

4. Experiments and Results

4.1 E1: Training duration

A 5.27M-parameter model trained on the initial 10,000-story pilot subset was evaluated after 500 and 5,000 updates. The 500-update run achieved validation loss 3.9758 in 28.74 seconds; the 5,000-update run achieved 2.5115 in 222.19 seconds. This is a 36.8% reduction in validation loss. It is exploratory evidence that additional optimisation improved predictive performance. However, the runs used different warmup lengths and cosine schedules (50 versus 200 warmup updates) and different validation sampling procedures. They are not a strictly controlled experiment isolating training duration.

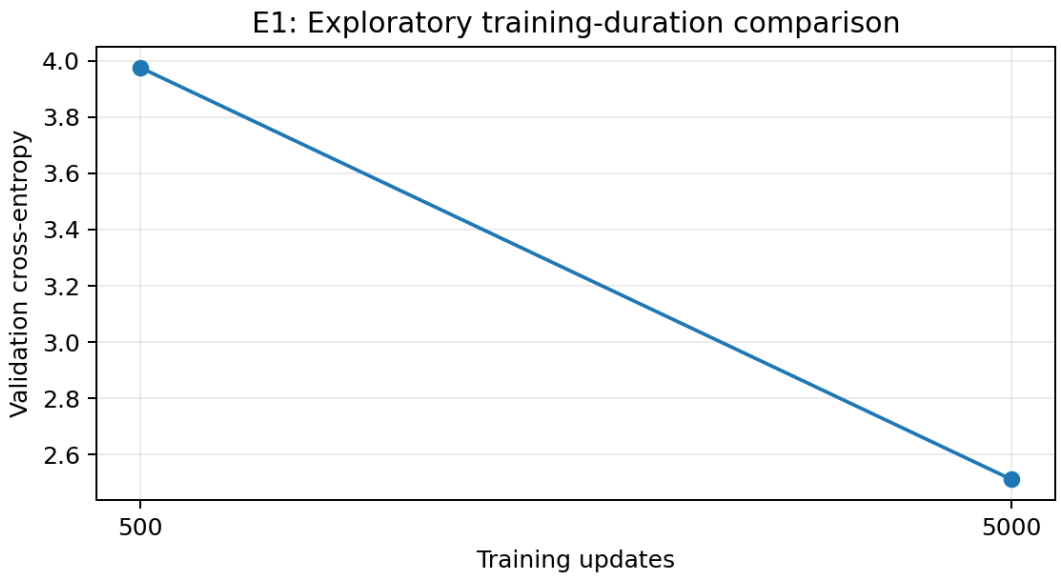


Figure 1. Validation loss in the exploratory 500- versus 5,000-update comparison. The runs differ in schedule and evaluation procedure.

4.2 E2: Training data quantity

E2 compared nested datasets of 10,000, 50,000, and 100,000 stories. Each configuration was trained for 5,000 updates with batch size 32 and context length 256. Three seeds (1234, 2027, 2028) were run per condition, using the same fixed validation batches.

Dataset	Stories	Mean val. loss	SD	Mean PPL
D1	10,000	2.5555	0.0072	12.8777
D2	50,000	2.5160	0.0021	12.3786
D3	100,000	2.5150	0.0086	12.3672

Table 1. E2 validation results across three random seeds (mean and sample standard deviation).

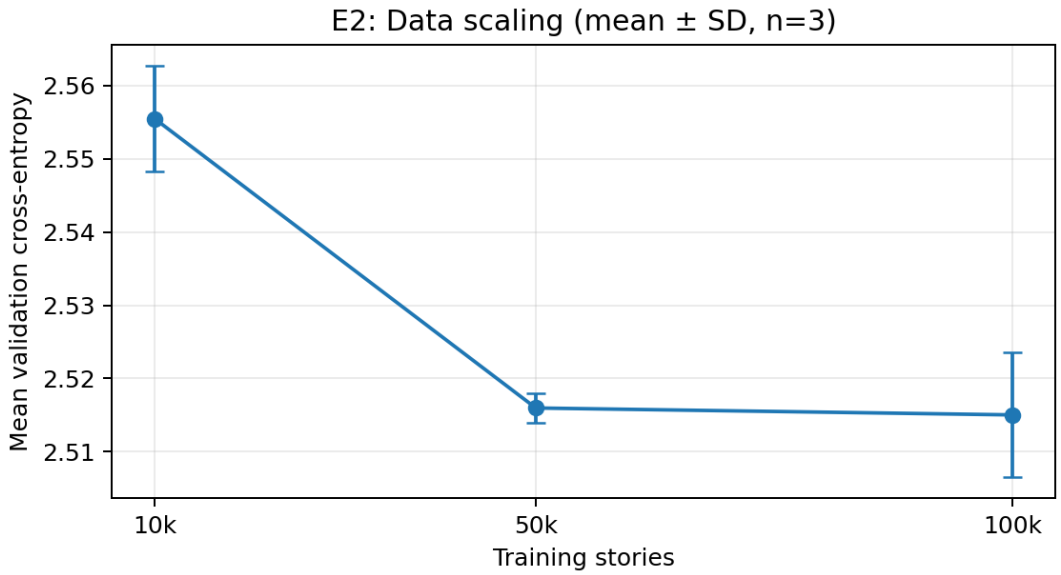


Figure 2. E2 validation loss by training-set size; error bars show sample standard deviation across three seeds.

Increasing the dataset from 10,000 to 50,000 stories reduced mean validation loss by approximately 0.0395. Increasing it from 50,000 to 100,000 reduced mean loss by only approximately 0.0009. The D2–D3 difference is small relative to observed variation across seeds. This is consistent with diminishing returns under a fixed 5,000-update budget, but three seeds provide limited precision and do not establish statistical significance.

Mean training time for replicated runs was approximately 212.3 seconds for D1, 210.8 seconds for D2, and 210.8 seconds for D3. Peak allocated GPU memory was approximately 1.44 GB across conditions. Because model and update count were fixed, larger datasets changed the sampling pool rather than the computation per update. With a fixed update count, larger datasets also receive fewer repeated exposures on average.

4.3 E3: Model capacity under a fixed wall-clock budget

E3 compared three model sizes on the 100,000-story training subset. Each model was trained for an approximately 150-second wall-clock allowance with the same batch size, context length, optimiser, and validation batches. Configurations were 4 layers/256 embedding/4 heads, 6/384/6, and 8/512/8.

Model	Parameters	Updates	Tokens	Val. loss	Peak GB
Small	5.27M	3,517	28.81M	2.4671	1.46
Medium	13.82M	1,645	13.48M	2.6092	2.00
Large	29.45M	903	7.40M	2.8401	2.79

Table 2. E3 model-scaling results. Each configuration was run once.

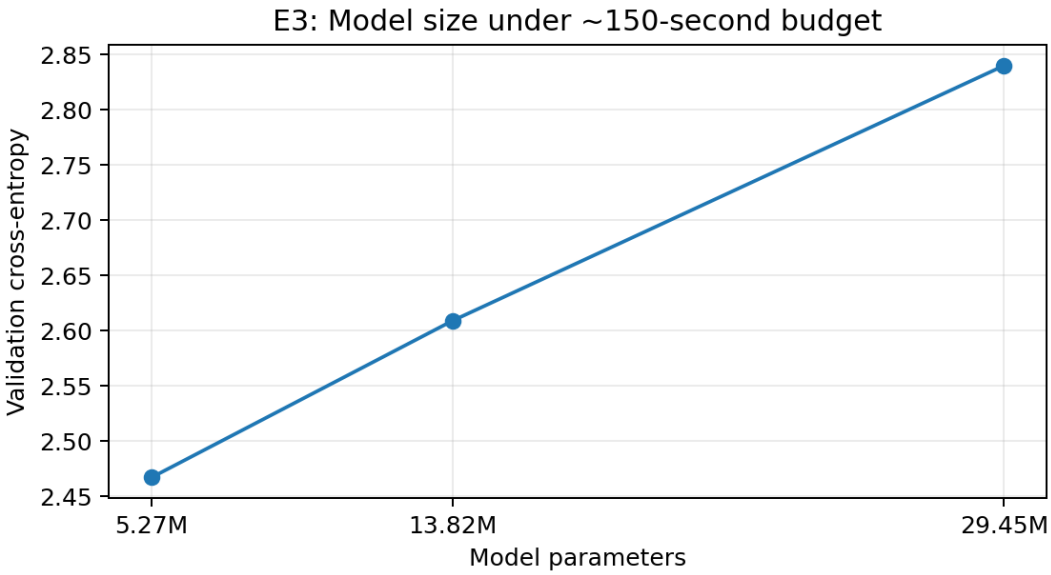


Figure 3. Validation loss against parameter count under an approximately 150-second budget.

The smallest model achieved the lowest validation loss. As model size increased, completed updates and processed token positions decreased, while peak memory increased. Under this T4 hardware and short budget, the smaller model used the available time more effectively. This does not show that smaller models are generally superior: the comparison fixes elapsed time rather than updates or tokens, and each size was tested with one seed. The fixed 100-update warmup also occupied a larger fraction of the available time for larger models.

5. Discussion

Across the three investigations, performance depended on how the limited training budget was allocated. E1's pilot comparison showed a large improvement with more updates, although schedule and evaluation differences limit causal interpretation. E2 provides the most controlled evidence: increasing data from 10,000 to 50,000 stories improved validation loss across three seeds, while the further increase to 100,000 yielded almost no additional gain under 5,000 updates. This may reflect diminishing returns, or that the fixed number of updates was insufficient to exploit the additional data.

E3 shows the cost of increasing model capacity under a wall-clock constraint. Larger models consumed more memory and completed fewer updates and token positions, and had higher validation loss. In this setting, additional capacity did not compensate for reduced optimisation progress. This finding is specific to the architecture, T4 GPU, short training duration, batch size, sequence length, and learning-rate schedule.

A practical implication is that model size, data quantity, and training duration should be considered jointly. A larger corpus is not automatically useful if the model cannot see enough of it during training, and a larger model may be disadvantaged when its per-update cost sharply reduces the number of updates possible. These results align with the motivation of compute-optimal training, but the small number of configurations and limited budget do not support fitting a scaling law.

Qualitative generations from the 5,000-update baseline contained recognisable story openings, dialogue, and moral-like endings, but also semantic contradictions, object confusion, and character inconsistency. This suggests that low validation loss alone does not guarantee coherent long-form generation. The examples were illustrative rather than scored by a blinded human evaluation, so no quantitative claim about generation quality is made.

6. Limitations

Several limitations qualify the findings. First, E1 was exploratory and changed the learning-rate schedule and validation sampling between runs. Second, E2 used three seeds and one fixed set of validation batches; standard deviations measure variation across training seeds conditional on those batches, not uncertainty over the full validation distribution. Third, E3 used one seed per model size and a short wall-clock budget, so its ranking may be sensitive to initialisation and warmup. Fourth, E3's measured elapsed time includes final evaluation overhead and can slightly exceed the nominal 150-second training-loop budget. Fifth, the reserved test split was not evaluated before submission; all reported predictive metrics are validation metrics. Sixth, only one dataset domain, tokenizer, context length, optimiser, and hardware type were investigated. Finally, model sizes and budgets are too limited to infer general scaling laws.

Future work should repeat E3 across multiple seeds, compare models under equal token and equal update budgets as well as equal time, evaluate on the untouched test set, test longer schedules, and investigate optimiser and context-length choices. Human or rubric-based evaluation of generated stories would complement token-level loss.

7. Conclusion

This project implemented and trained small decoder-only Transformer language models from random initialisation on TinyStories using a Tesla T4 GPU. The experiments indicate that longer training can improve validation performance, that increasing data from 10,000 to 50,000 stories can help under a fixed update budget while further gains diminish, and that a smaller model can outperform larger alternatives when elapsed training time is constrained. The findings are local to the experimental setup and should not be interpreted as universal prescriptions. Overall, compute-efficient language-model training involves balancing model capacity, data diversity, and optimisation progress.

AI Assistance Declaration

Generative AI tools, including ChatGPT (OpenAI), were used to assist with research ideation, experimental planning, PyTorch implementation, code debugging, interpretation of experimental results, and editing and drafting of the report. All experiments were executed by the author using Google Colab and a NVIDIA Tesla T4 GPU. The author is responsible for reviewing and verifying the implementation, experimental results, references, and final submitted work, and for ensuring an understanding of the methods and conclusions presented.

References

[1] J. Hoffmann et al. (2022). Training Compute-Optimal Large Language Models. *Advances in Neural Information Processing Systems*, 35.

[2] R. Eldan and Y. Li (2023). TinyStories: How Small Can Language Models Be and Still Speak Coherent English? arXiv:2305.07759.

[3] A. Vaswani et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.

[4] R. Sennrich, B. Haddow, and A. Birch (2016). Neural Machine Translation of Rare Words with Subword Units. *Proceedings of ACL*.

[5] N. Muennighoff et al. (2023). Scaling Data-Constrained Language Models. *Advances in Neural Information Processing Systems*, 36.